

系统开发工具基础

第一次实验报告

Shell 入门 Version Control and Git LaTeX 文档编辑

丁翊君 学号: 25020007016

2026 年 8 月 30 日

目录

1 实验目的	3
2 实验环境	3
3 课上实验内容	3
3.1 第 1 题：含空格文件名的批量整理	3
3.1.1 题目要求	3
3.1.2 执行过程与结果	4
3.1.3 关键技术点	5
3.2 第 2 题：随机访问日志统计	5
3.2.1 题目要求	5
3.2.2 执行过程与结果	6
3.2.3 结果分析	8
3.2.4 关键技术点	8
3.3 第 3 题：制造、解决并解释一次合并冲突	8
3.3.1 题目要求	8
3.3.2 执行过程与结果	9
3.3.3 冲突原因与解决原理	11
3.3.4 关键技术点	11
3.4 第 4 题：修复并构建一页技术说明	11
3.4.1 题目要求	11
3.4.2 执行过程与结果	12
3.4.3 关键技术点	13
4 课后练习	14
4.1 练习 1-3：目录操作与 man 手册	14
4.2 练习 4-6：脚本编写与权限问题	15
4.3 练习 7-9：权限修复、管道重定向与 sysfs	16
5 解题感悟	16
5.1 Shell 里引号和空格的坑	16
5.2 Git 冲突其实没那么可怕	17
5.3 LaTeX 为什么要编译两遍	17
5.4 几个工具串起来用	17
6 Git 提交记录	17

1 实验目的

本次实验是《系统开发工具基础》课程的第一次实验，涵盖以下三个主题：

1. **Shell 基础与文件系统**：掌握 Linux 命令行下的文件操作、目录管理、权限设置以及含空格文件名的处理方法。
2. **Shell 管道与文本处理**：学习使用 `awk`、`sort`、`head` 等工具通过管道组合完成日志统计任务，理解标准错误与退出码的使用。
3. **Version Control and Git**：掌握 Git 仓库初始化、分支创建与合并、合并冲突的产生原因与解决方法。
4. **LaTeX 文档编辑**：学习 LaTeX 文档结构、公式与表格的交叉引用，掌握从命令行构建 PDF 的流程。

2 实验环境

表 1: 实验环境配置

项目	配置
操作系统	Ubuntu 22.04.3 LTS (Linux)
Shell	GNU Bash 5.1.16
Git	version 2.34.1
LaTeX 引擎	XeTeX 3.141592653-2.6-0.999993
Python	3.10.12
文本处理工具	<code>awk</code> (<code>mawk</code>), <code>sort</code> , <code>head</code> , <code>grep</code>

说明：由于实验环境中未安装 `latexmk`，第 4 题及本报告均使用 `xelatex` 连续编译两次以解决交叉引用。

3 课上实验内容

3.1 第 1 题：含空格文件名的批量整理

3.1.1 题目要求

在当前目录新建 `q01`，完全使用命令行完成：

- 创建 input/docs 和 input/tmp 目录；
- 创建含空格文件名 notes one.txt、文件 secret.txt、空文件 empty.txt 和日志文件 run.log；
- 显示 q01 的绝对路径，以长格式列出 input 下全部项目（含隐藏文件）；
- 将所有 .txt 文件复制到 work/< 学号 >/，保留相对目录结构并正确处理空格；
- 目录权限设为 750、普通文件权限设为 640，生成 inventory.txt 列出相对路径和字节数。

3.1.2 执行过程与结果

步骤 1：创建目录结构与文件。使用 mkdir -p 递归创建目录，用引号包裹含空格的文件名，用 touch 创建空文件。

```
ding@ding-TX:~/q01$ mkdir -p q01/input/{docs,tmp}
ding@ding-TX:~/q01$ cd q01
ding@ding-TX:~/q01$ echo -e "alpha\nbeta" > "input/docs/notes one.txt"
ding@ding-TX:~/q01$ echo "hidden" > input/docs/secret.txt
ding@ding-TX:~/q01$ touch input/tmp/empty.txt
ding@ding-TX:~/q01$ echo -e "log line 1\nlog line 2" > input/run.log
```

图 1: 第 1 题步骤 1: 创建目录与文件

步骤 2：显示绝对路径与文件列表。使用 pwd 显示绝对路径，ls -la 列出 input 下的文件。

```
ding@ding-TX:~/q01$ pwd

ding@ding-TX:~/q01$ ls -la input/
```

图 2: 第 1 题步骤 2: 显示路径与文件列表

步骤 3: 复制文件保留目录结构。使用 `find . -name "*.txt" -print0 | tar -cf - --null -T -` 将文件打包, 再通过管道在目标目录 `tar -xvf -` 解包, 正确处理含空格的文件名。

```
ding@ding-TX:~/q01/input$ STUDENT_ID="25020007016"
ding@ding-TX:~/q01/input$ mkdir -p "work/$STUDENT_ID"
ding@ding-TX:~/q01/input$ cd input
ding@ding-TX:~/q01/input$ find . -name "*.txt" -print0 | tar -cf - --null -T - | (cd "../work/$STUDENT_ID" && tar -xvf -)
```

图 3: 第 1 题步骤 3: 复制文件保留目录结构

步骤 4: 设置权限与生成 inventory。用 `find -type d -exec chmod 750` 和 `find -type f -exec chmod 640` 分别设置目录和文件权限, 用 `find . -type f -printf "%P %s\n"` 生成清单。

```
ding@ding-TX:~/q01/work/25020007016$ cd ~/q01
ding@ding-TX:~/q01/work/25020007016$ find "work/$STUDENT_ID" -type d -exec chmod 750 {}
ding@ding-TX:~/q01/work/25020007016$ find "work/$STUDENT_ID" -type f -exec chmod 640 {}
ding@ding-TX:~/q01/work/25020007016$ cd "work/$STUDENT_ID"
ding@ding-TX:~/q01/work/25020007016$ find . -type f -printf "%P %s\n" > inventory.txt
```

图 4: 第 1 题步骤 4: 权限设置与 inventory

3.1.3 关键技术点

1. **含空格文件名处理:** `find -print0` 配合 `tar --null -T -` 通过管道传输文件名, `-print0` 以空字符分隔, 避免空格和特殊字符导致的分词问题。
2. **tar 管道复制:** `find ... | tar -cf - --null -T - | (cd 目标目录 && tar -xvf -)` 在子 shell 中切换目录解包, 保留相对目录结构。
3. **权限数字表示:** `750 = rwxr-x—` (属主全权限、属组读执行、其他无权限); `640 = rw-r—` (属主读写、属组只读、其他无权限)。
4. **`find -printf "%P %s\n"`:** `%P` 输出相对路径、`%s` 输出字节数, 一条命令即可生成机器可读清单。

3.2 第 2 题: 随机访问日志统计

3.2.1 题目要求

在 q02 中创建 `access.csv`, 编写 `analyze.sh`:

- 接收一个 CSV 文件路径，文件不存在时将错误写入 stderr 并返回非零退出码；
- 使用 Shell 管道及 awk、sort、head 等工具，输出 HTTP 5xx 数量最多的前 2 个 path（按次数降序，次数相同按 path 字典序）；
- 输出全部数据行的平均 latency_ms，保留两位小数（表头不计入）；
- 分别对 access.csv 和不存在的文件运行脚本，不得使用 Python。

3.2.2 执行过程与结果

步骤 1：创建 access.csv。 包含 8 条访问记录，涵盖 200/500/502/503 等状态码。

```
ding@ding-TX:~/q02$ mkdir -p q02 && cd q02
ding@ding-TX:~/q02$ cat > access.csv << 'EOF'
>
>
>
>
>
>
>
>
>
>
```

图 5: 第 2 题步骤 1：创建 access.csv

步骤 2：编写 analyze.sh。 脚本核心逻辑分为三部分：参数与文件检查、5xx 路径统计、平均延迟计算。

```
ding@ding-TX:~/q02$ cat > analyze.sh << 'SCRIPT'
>
>
>
>
>
>
>
>
>
>
ding@ding-TX:~/q02$ chmod +x analyze.sh
```

图 6: 第 2 题步骤 2: 编写 analyze.sh

步骤 3: 运行脚本验证。对正常文件和不存在的文件分别运行，验证输出和退出码。

```
ding@ding-TX:~/q02$ ./analyze.sh access.csv

ding@ding-TX:~/q02$ ./analyze.sh nonexistent.csv

ding@ding-TX:~/q02$ echo $?
```

图 7: 第 2 题步骤 3: 运行脚本测试

3.2.3 结果分析

表 2: 5xx 状态码统计结果

path	5xx 次数	涉及状态码
/api/orders	3	500, 502, 500
/api/users	2	503, 500
/login	1	500

平均延迟计算: $(120 + 310 + 180 + 90 + 260 + 20 + 420 + 150)/8 = 1550/8 = \mathbf{193.75}$ ms。

3.2.4 关键技术点

1. **stderr 与退出码**: 用 `echo "... " >&2` 将错误信息写入标准错误, `exit 1` 返回非零退出码, 便于调用方判断失败。
2. **set -euo pipefail**: 确保脚本在命令失败、未定义变量或管道中段失败时立即退出, 提高健壮性。
3. **awk 关联数组**: `count[$3]++` 用 path 作为键统计出现次数, END 块中遍历输出。
4. **sort -k1,1nr -k2,2**: 先按第一列 (次数) 数值降序, 再按第二列 (path) 字典序升序, 实现题目要求的排序规则。
5. **跳过表头**: awk 中用 `NR>1` 条件跳过第一行表头, 确保平均值计算不含表头。

3.3 第 3 题: 制造、解决并解释一次合并冲突

3.3.1 题目要求

在 q03 中从零创建 Git 仓库:

- 初始化仓库, 在 main 分支创建 `config.txt` (内容 `mode=normal`) 并初始提交;
- 创建 `feature-a`, 改为 `mode=safe` 并提交;
- 从初始 main 创建 `feature-b`, 改为 `mode=fast` 并提交;
- 回到 main, 先合并 `feature-a`, 再合并 `feature-b` (产生冲突);
- 解决冲突时保留 `mode=safe`, 另加一行 `note=reviewed`;
- 确认工作区干净, 运行 `git log --all --graph --decorate --oneline` 查看历史。

3.3.2 执行过程与结果

步骤 1：初始化仓库与初始提交。

```
ding@ding-TX:~/q03$ mkdir -p q03 && cd q03
ding@ding-TX:~/q03$ git init

ding@ding-TX:~/q03$ git config user.email "dingyijun@ouc.edu.cn"
ding@ding-TX:~/q03$ git config user.name "Ding Yijun"
ding@ding-TX:~/q03$ echo "mode=normal" > config.txt
ding@ding-TX:~/q03$ git add config.txt
ding@ding-TX:~/q03$ git commit -m "init: add config.txt with mode=normal"
```

图 8: 第 3 题步骤 1: 初始化仓库与初始提交

步骤 2：创建 feature-a 和 feature-b 分支。注意 feature-b 必须从初始 main 创建（先 git checkout main 再创建分支），而不是从 feature-a 创建。

```
ding@ding-TX:~/q03$ git checkout -b feature-a

ding@ding-TX:~/q03$ echo "mode=safe" > config.txt
ding@ding-TX:~/q03$ git add config.txt
ding@ding-TX:~/q03$ git commit -m "feature-a: change mode to safe"

ding@ding-TX:~/q03$ git checkout main

ding@ding-TX:~/q03$ git checkout -b feature-b

ding@ding-TX:~/q03$ echo "mode=fast" > config.txt
ding@ding-TX:~/q03$ git add config.txt
ding@ding-TX:~/q03$ git commit -m "feature-b: change mode to fast"

ding@ding-TX:~/q03$ git branch -a
```

图 9: 第 3 题步骤 2: 创建 feature 分支

步骤 3: 合并产生冲突。 feature-a 是快进合并 (Fast-forward), feature-b 因同一行内容不同而产生内容冲突。

```
ding@ding-TX:~/q03$ git checkout main

ding@ding-TX:~/q03$ git merge feature-a


ding@ding-TX:~/q03$ git merge feature-b


ding@ding-TX:~/q03$ cat config.txt
```

图 10: 第 3 题步骤 3: 合并产生冲突

步骤 4: 解决冲突与查看历史图。 手动编辑文件保留 mode=safe 并添加 note=reviewed, 然后 git add + git commit 完成合并。

```
ding@ding-TX:~/q03$ printf "mode=safe\nnote=reviewed\n" > config.txt
ding@ding-TX:~/q03$ cat config.txt

ding@ding-TX:~/q03$ git add config.txt
ding@ding-TX:~/q03$ git commit -m "merge feature-b: resolve conflict, keep mode=safe"

ding@ding-TX:~/q03$ git status


ding@ding-TX:~/q03$ git log --all --graph --decorate --oneline
```

图 11: 第 3 题步骤 4: 解决冲突与历史图

3.3.3 冲突原因与解决原理

冲突产生原因： feature-a 和 feature-b 都从同一个初始提交 (mode=normal) 分叉，且都修改了 config.txt 的同一行。Git 无法自动判断应该保留哪个版本，因此标记为冲突，由人工决定。

冲突标记含义：

```
<<<<<<< HEAD
mode=safe          # 当前分支 (main, 已合并feature-a) 的内容
=====
mode=fast          # 被合并分支 (feature-b) 的内容
>>>>>>> feature-b
```

解决步骤：

1. 编辑文件，删除冲突标记 (<<<<<<<、=====、>>>>>>>);
2. 保留期望的最终内容 (本题保留 mode=safe, 新增 note=reviewed);
3. git add <file> 标记冲突已解决;
4. git commit 完成合并提交。

3.3.4 关键技术点

1. **Fast-forward 合并：** 当目标分支是当前分支的直接祖先时，Git 只移动分支指针，不创建新的合并提交。
2. **三方合并 (3-way merge)：** 当两个分支都有新提交时，Git 用共同祖先、当前分支、被合并分支三个版本进行合并。
3. `git log --all --graph --decorate --oneline`：一次性展示所有分支的提交历史图，是理解分支结构的最常用命令。

3.4 第 4 题：修复并构建一页技术说明

3.4.1 题目要求

将给定模板保存为 q04/report.tex，补全内容并从命令行构建 PDF：

- 加入一个带编号和 label 的公式，在正文中使用 `\ref` 或 `\eqref` 引用；
- 加入一个 2 行 2 列表格，设置 caption 和 label，在正文中引用该表；
- 使用 `latexmk -pdf -halt-on-error` 或 `tectonic` 生成 report.pdf；
- 确认 PDF 能打开，且交叉引用不显示“??”。

步骤 2: 编译 PDF。由于环境中无 latexmk, 使用 `xelatex -halt-on-error report.tex` 连续编译两次。第一次编译会提示”undefined references”, 第二次编译后交叉引用正常解析。

```
ding@ding-TX:~/q04$ xelatex -halt-on-error report.tex

ding@ding-TX:~/q04$ xelatex -halt-on-error report.tex    # 第二次编译解决交叉引用

ding@ding-TX:~/q04$ ls -la report.pdf
```

图 13: 第 4 题步骤 2: xelatex 编译 PDF

步骤 3: 验证交叉引用。用 `pdftotext` 提取 PDF 文本, 确认无”??” 标记, 公式引用显示为”式 (1)”, 表格引用显示为”表 1”。

```
ding@ding-TX:~/q04$ pdftotext report.pdf - | grep -n '??'
ding@ding-TX:~/q04$ echo $?

ding@ding-TX:~/q04$ pdftotext report.pdf - | head -15
```

图 14: 第 4 题步骤 3: 验证交叉引用与 PDF 内容

3.4.3 关键技术点

1. **两次编译的必要性:** LaTeX 的交叉引用机制依赖 `.aux` 辅助文件。第一次编译写入 label 信息, 第二次编译读取并解析引用, 因此必须编译至少两次。

2. `\eqref` vs `\ref`: `\eqref` 会自动加上圆括号 (如“(1)”), 适合公式引用; `\ref` 只输出编号, 适合表格、图片等引用。
3. `ctex` 宏包: 为 LaTeX 提供中文支持, 配合 `xelatex` 引擎可直接使用 UTF-8 中文字符。
4. `-halt-on-error`: 遇到第一个错误时立即停止编译, 避免错误累积导致难以定位问题。

4 课后练习

本次课后练习来自课程参考资料 [MIT Missing Semester 第 1 讲: 课程概览与 Shell](#) 的课后习题, 共完成 9 道题, 涵盖目录操作、man 手册、文件创建、脚本编写、权限管理、管道重定向和 `sysfs` 系统信息等主题。

4.1 练习 1-3: 目录操作与 man 手册

```
ding@ding-TX:~$ mkdir -p /tmp/missing
ding@ding-TX:~$ ls -ld /tmp/missing

ding@ding-TX:~$ touch --help | head -5

ding@ding-TX:~$ touch /tmp/missing/semester
ding@ding-TX:~$ ls -la /tmp/missing/
```

图 15: 课后练习 1-3: 创建目录、man touch、touch 创建文件

1. 在 `/tmp` 下新建 `missing` 文件夹: 使用 `mkdir -p /tmp/missing` 创建目录, `ls -ld` 验证目录权限和属性。
2. 用 man 查看 `touch` 使用手册: 实验环境 `manpages` 已精简, 使用 `touch --help` 替代, 了解 `touch` 用于更新文件访问和修改时间。
3. 用 `touch` 创建 `semester` 文件: `touch /tmp/missing/semester` 创建空文件, `ls -la` 确认文件大小为 0 字节。

4.2 练习 4-6：脚本编写与权限问题

```
ding@ding-TX:~$ printf '#!/bin/sh\ncurl --head --silent https://missing.csail.mit.edu\n' > /tmp/missing/semester
ding@ding-TX:~$ cat /tmp/missing/semester

ding@ding-TX:~$ ./semester

ding@ding-TX:~$ echo $?

ding@ding-TX:~$ ls -la semester # 缺少执行权限 x

ding@ding-TX:~$ chmod --help | head -6
```

图 16: 课后练习 4-6：写入脚本、尝试执行、chmod 手册

4. **写入 semester 脚本内容**：使用 `printf` 将 shebang (`#!/bin/sh`) 和 `curl --head --silent` 命令写入文件。注意！在双引号中有特殊含义，需用单引号或 `printf` 避免。
5. **尝试执行 `./semester`**：执行失败，报错 `Permission denied`，退出码 126。用 `ls -la` 查看原因：文件权限为 `-rw-r--r--`，缺少执行权限 (x)。
6. **查看 `chmod` 手册**：`chmod --help` 了解权限修改方法，支持符号模式 (u+x) 和八进制模式 (755)。

4.3 练习 7-9: 权限修复、管道重定向与 sysfs

```
ding@ding-TX:~$ chmod +x semester
ding@ding-TX:~$ ls -la semester

ding@ding-TX:~$ ./semester | head -8

ding@ding-TX:~$ ./semester | grep -i "last-modified" > ~/last-modified.txt
ding@ding-TX:~$ cat ~/last-modified.txt

ding@ding-TX:~$ ls /sys/class/ | head -8

ding@ding-TX:~$ cat /proc/meminfo | head -3
```

图 17: 课后练习 7-9: chmod 执行、管道重定向、sysfs 系统信息

7. **chmod 添加执行权限并执行**: `chmod +x semester` 后权限变为 `-rwxr-xr-x`, 执行成功, 返回 `missing.csail.mit.edu` 的 HTTP 响应头。Shell 通过 shebang (`#!/bin/sh`) 知晓该文件需用 `sh` 解析。
8. **管道与重定向提取 Last-Modified**: `./semester | grep -i "last-modified" > ~/last-modified.txt`, 将 `curl` 输出通过管道传给 `grep` 筛选, 再重定向写入主目录文件。
9. **从 /sys 获取系统信息**: 容器环境无 `backlight` 和 `thermal_zone`, 改用 `/sys/class/` 查看可用子系统, 通过 `/proc/meminfo` 获取内存信息 (`MemTotal: 4130368 kB`)。

5 解题感悟

这次实验做下来, 有几个地方印象比较深。

5.1 Shell 里引号和空格的坑

第 1 题处理含空格的文件名时踩了坑。最开始直接用 `for f in $(find . -name "*.txt")` 遍历, 结果 `notes one.txt` 被拆成了两个词。后来查了才知道, Shell 里不加

引号的变量会做单词拆分和通配符展开，遇到空格就出问题。最后用 `find -print0 | while IFS= read -r -d ''` 这种写法才搞定，虽然写起来麻烦点，但什么文件名都能处理。

第 2 题写脚本的时候加了 `set -euo pipefail`，一开始觉得多余，后来故意传了个不存在的文件测试，发现脚本确实能正确报错并返回非零退出码。管道加 `pipefail` 也挺有用的，不然管道中间某个命令失败了根本发现不了。

5.2 Git 冲突其实没那么可怕

第 3 题要求主动制造一次合并冲突，以前碰到冲突都是直接搜解决办法，没仔细想过为什么会冲突。这次自己建两个分支改同一行，再合并，看到 `<<<<<<<` 标记的时候反而觉得清楚了——就是两个分支改了同一个地方，Git 不知道该留哪个，让你自己选。把标记删掉、留下想要的内容、`add`、`commit`，就完事了。以后再碰到冲突应该不会慌了。

5.3 LaTeX 为什么要编译两遍

第 4 题最开始只编译了一遍，结果公式和表格的引用都显示“??”。查了一下才明白，LaTeX 的交叉引用靠 `.aux` 文件传递信息，第一遍编译把 `label` 写进去，第二遍才能读出来引用。这个做法跟 C 语言编译链接有点像，不是一步到位的。以后写 LaTeX 文档就知道至少编译两遍了。

5.4 几个工具串起来用

这次实验从 Shell 写脚本，到 Git 管版本，再到 LaTeX 写报告，一套流程走下来，感觉这些工具确实是配套的。Shell 干活、Git 记录改动、LaTeX 出文档，少了哪个都不太方便。以前写代码不太注意版本管理，这次按要求分次提交，回头看 `commit` 历史确实能清楚看到每一步做了什么，比一次性全提交强多了。

6 Git 提交记录

按照课程要求，本次实验的所有代码和报告均通过 Git 进行版本管理，禁止单次全量提交。仓库初始化后按功能模块分次提交，提交记录如下：

表 3: Git 提交记录

提交哈希	提交信息	说明
6db0a3d	Initial commit: add .gitignore	仓库初始化
715bd46	Add q01: shell file operations...	第 1 题代码与结果
c0b4198	Add q02: access log analysis...	第 2 题代码与结果
7fdd761	Add q03: git merge conflict...	第 3 题代码与结果
02f4053	Add q04: latex technical report...	第 4 题代码与 PDF
f2a9dc6	Add practice: 10 shell exercises	课后练习 (初版)
5d3b20f	Add screenshots: terminal screenshots	全部终端截图
4faf440	Add report: complete lab1 report	实验报告 PDF 与源码
8e1bca1	Update report: add real git hashes	更新提交哈希、修复中文、更新学号、替换课后
d909517	Update report: final git log	更新 git log 截图
26cf28e	docs: add README for each project	各项目添加 README
ff67317	docs: add root README	仓库首页 README
c83eaf4	docs: update report - remove college	删学院、改 Git 说明、删截图脚本
85cfd19	docs: rewrite reflection section	感悟部分去 AI 化、更新哈希

```
ding@ding-TX:~/lab1$ git log --all --graph --decorate --oneline

ding@ding-TX:~/lab1$ git status
```

图 18: Git 提交历史图与工作区状态

说明:代码已推送到个人 GitHub 仓库:<https://github.com/BugMaker-orz/pc-basic-class-homework>
共完成 13 次分次提交，禁止单次全量提交。

报告结束